

智能巡检车 PRO 版实验手册

雷达避障实战

目录

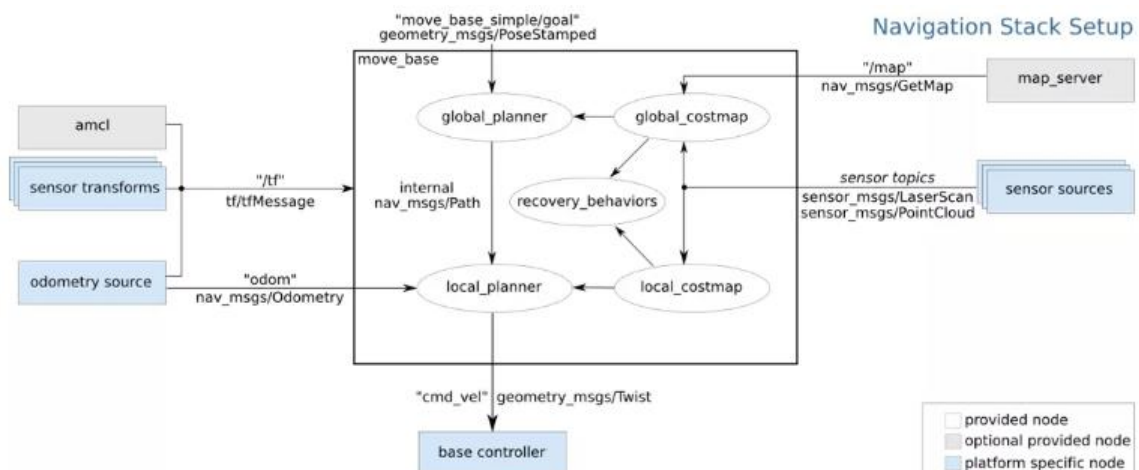
1 实验背景	2
2 实验原理	12
3 实验目的	13
4 实验环境	13
5 实验流程	14
6 实验任务	15
1.1.1 任务一 Navigation 导航仿真	15
1.1.2 任务二 Navigation 导航仿真遇到挡路障碍物	20
7 实验结果	29
8 实验资源释放	30
9 实验小结	30

1

实验背景

在前面课程中，我们在 ROS 机器人仿真环境中，使用 Gmapping 激光 SLAM 建图方法，获得了仿真环境的 2D 栅格地图，接下来我们将学习 ROS 中的 Navigation 导航系统，以及基于 2D 栅格地图的路径规划和导航仿真实验。

Navigation 导航系统是 ROS 中最常用的子系统，主要用于使机器人能够在已知或未知环境中自主移动，到达指定的目标位置。其系统架构图如下：



Navigation 的主要组成部分

global_costmap: 这个包负责创建和更新环境的代价，包括静态障碍物、动态障碍物以及机器人周围的通路信息。代价会根据激光雷达或其他传感器的数据实时更新，为路径规划提供必要的环境信息。

move_base: **move_base** 是 ROS 导航系统的核心节点，它集成了全局路径规划器、局部路径规划器以及控制器。它接收目标位置，利用全局路径规划器计算出从当前位置到目标位置的最佳路径，然后通过局部路径规划器和控制器来执行具体的移动操作，同时避免碰撞。

global_planner: 全局路径规划器负责生成从起始点到终点的最优路径。这个过程通常

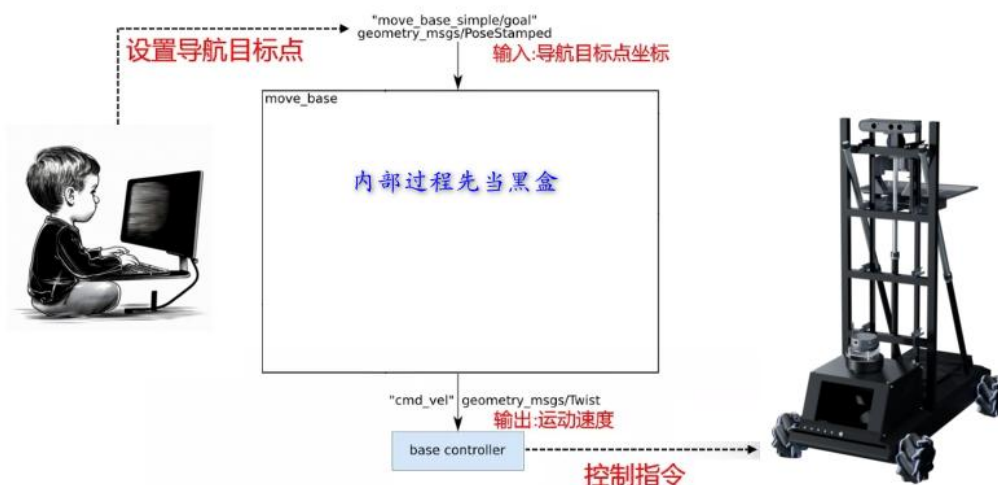
基于预先构建的地图进行，使用 A*算法或 Dijkstra 算法等搜索方法来寻找最短路径。

local_planner: 局部路径规划器关注的是机器人的即时行动，它会根据当前的速度、方向以及附近的障碍物情况来调整机器人的运动轨迹，确保机器人能够安全地沿着全局路径前进。

recovery_behaviors: 当机器人遇到无法解决的问题时（比如陷入死胡同），恢复行为可以帮助机器人摆脱困境。常见的恢复策略包括旋转、倒退等。

Navigation 如何使用？

我们都知道 Navigation 是用来导航的，但是它到底如何帮助我们进行导航呢？只要找出它的输入量和输出量就明白了。我们将 Navigation 的架构图简化，只保留输入量和输出部分，通过看清楚它和外界的对接关系来理解它是如何实现导航的。



从上图可以看出，Navigation 的输入是开发者设置的导航目标点坐标，输出是机器人的运动控制指令。我们只需要让机器人的主控节点订阅主题“cmd_vel”里的速度指令，并向导航服务“move_base_simple/goal”提交导航目标点就可以了。Navigation 会自动向话题“cmd_vel”发送速度值，驱动机器人到达导航目标。

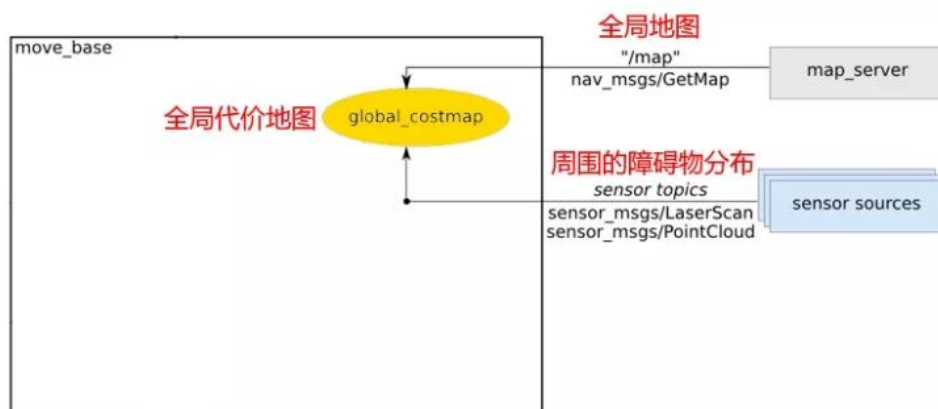
全局导航的路线怎么生成

要进行导航需要有地图，这个地图可以用 SLAM 建图方法来创建得到（参看上一节实

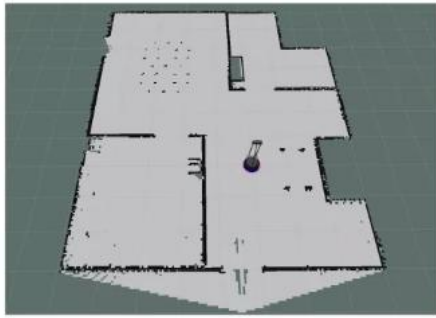
验)。但是原始的地图并不能直接进行导航，通常需要先将其转换成“代价地图”(cost_map)。

“代价地图”，就是说机器人在地图里移动是需要付出“代价”的，这个“代价”有显性的也有隐性的。显性的，比如行走的距离，绕远路费时费力，这是最明显的“代价”。隐性的，比如太靠近障碍物容易发生碰撞，这是隐性的“代价”。还有一些机器人类型比较长（比如：无人驾驶卡车），转弯掉头成本会比较麻烦，对它来说，路线转弯太多就意味着“代价”大，导航路线越顺滑则代价就越小。

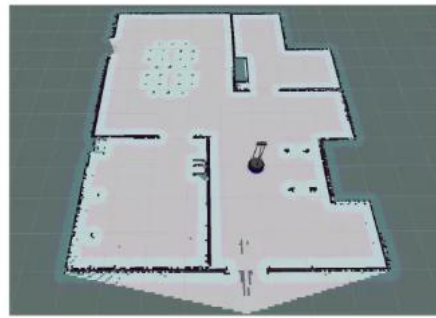
那么在 Navigation 系统里，“代价地图”是如何生成的呢？我们来看架构图的右上角部分。



可以看到，全局代价地图是通过 map_server 提供的全局地图（通常是 SLAM 建好的）和激光雷达侦测到的当前机器人周围的障碍物分布融合后生成的。map_server 提供的全局地图代表的是以前记录的地图，不排除在使用时可能会有变化。比如：路上多了个行人、曾经开着的门被关上了、路被障碍物给堵上了等。较远处的变化，机器人雷达扫描不到先不考虑，近处的变化可以用激光雷达侦测扫描，这就是为何全局代价地图需要融合两者的信息来生成。Navigation 生成的全局代价地图可以在 Rviz 里查看，比如下图：



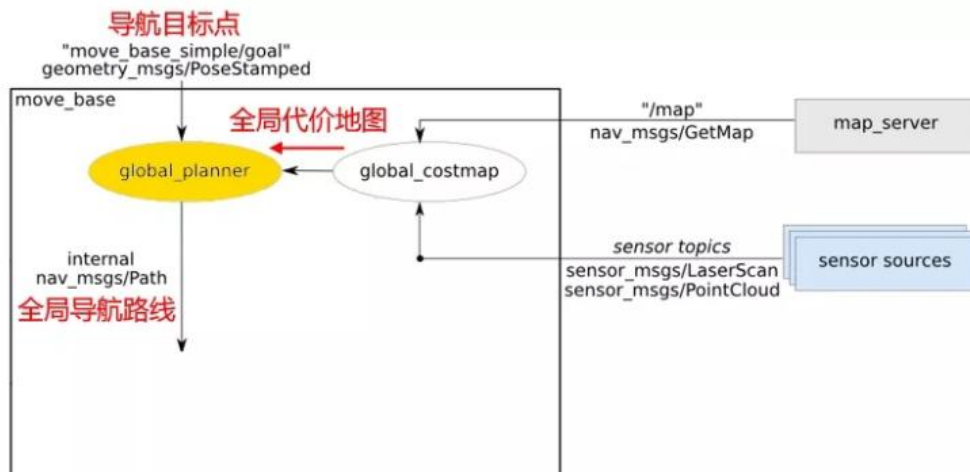
全局地图



全局代价地图

可以看到全局代价地图里，在障碍物的边缘会膨胀出一层淡蓝色的渐变区域，这个代表的就是机器人可能与障碍物发生碰撞的隐性“代价”区域。越靠近障碍物，与障碍物碰撞的风险越大，于是颜色越深，隐性“代价”越大。移动距离产生的显性代价，通常都是在路径规划算法内部才会实现，在 Rviz 里一般不会显示。

有了代价地图以后，如何得到导航的路线？在 Navigation 系统里，这个通常由全局规划器（global_planner）来生成。

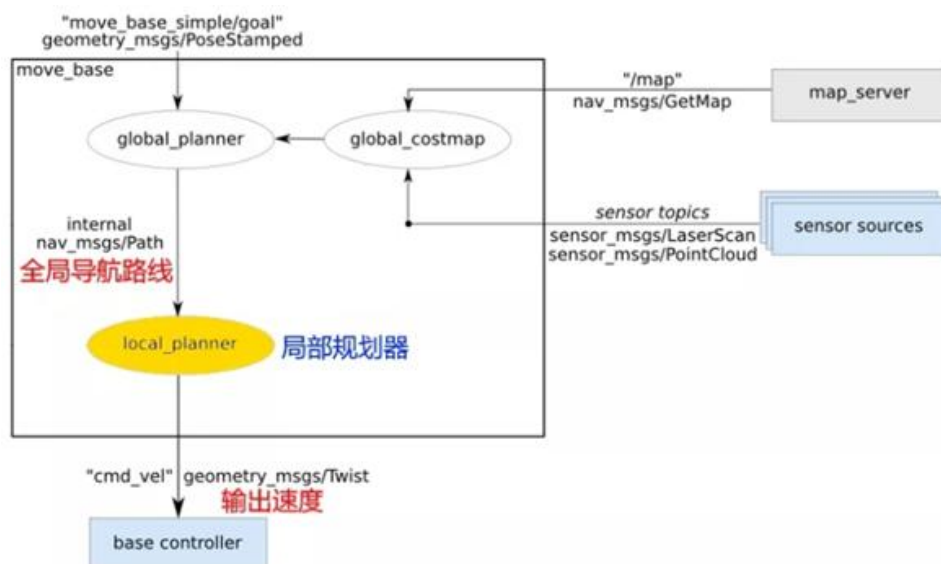


从上图可以看出，全局规划器的任务就是从外部获得导航的目标点，然后在全局代价地图里找出“代价最小”的那条路线，这条路线就是我们要找的全局导航路线。

局部规划器

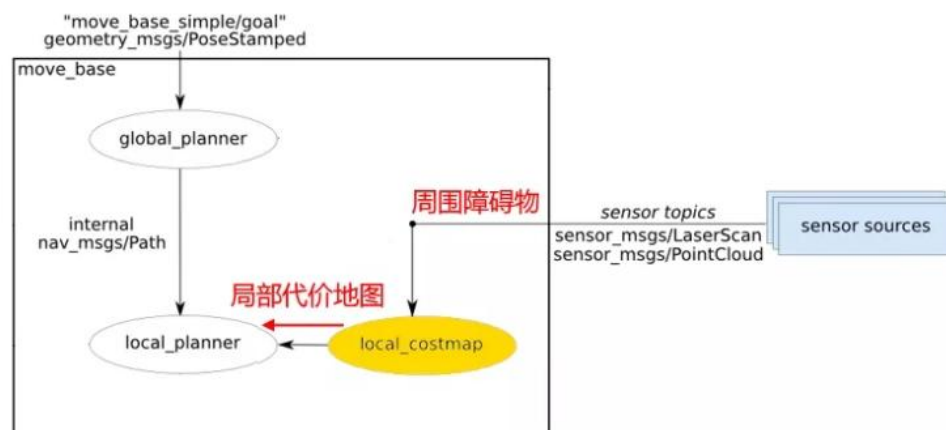
有了全局规划路线，并不意味着就能实现导航了，因为路线上的情况可能会发生变化，比如随时出现的行人和障碍物等。这就需要局部规划器（local_planner）去随机应变地处理。

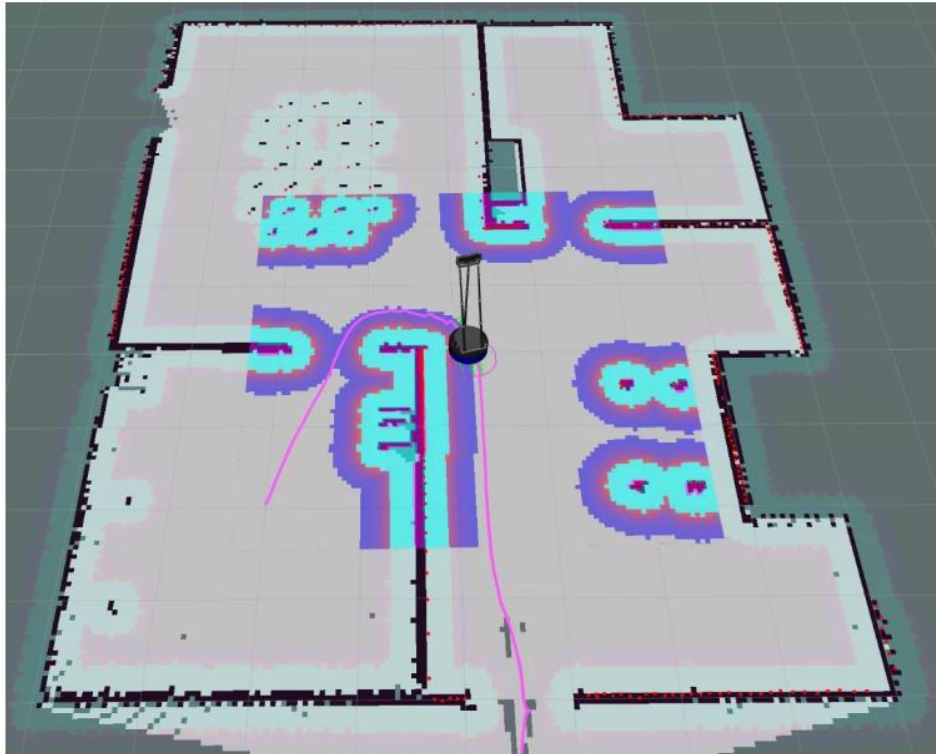
理这些突发情况。局部规划器从全局规划器获得导航路线，根据这个路线向机器人发送速度，然后一边往全局路线总体方向去走，一边根据突发情况做必要的临时修正，驱动机器人“尽可能”地按照路线去顺利到达目标点。



局部代价地图

局部规划器为了完成任务目标，利用激光雷达获得的当前障碍物数据，又生成了一个小范围的“代价地图”，叫做局部代价地图（local_costmap）。在机器人移动过程中，让局部代价地图跟着走，始终围绕在机器人周围，从而弥补了全局代价地图里无法体现随时移动障碍物的缺点。





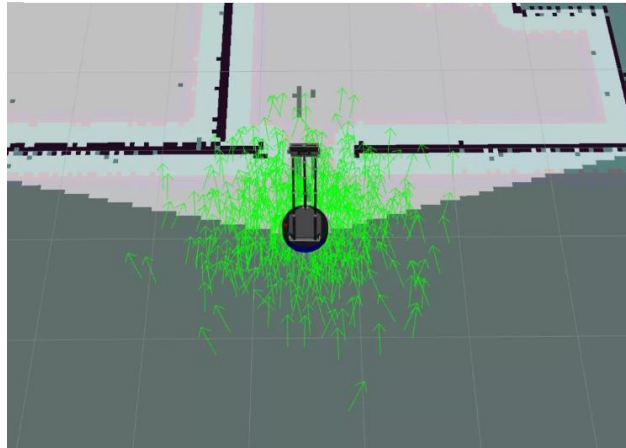
Navigation 这一设计考虑了实际情况，在坚持大方向正确的前提下，兼顾到局部具体细节处理，为机器人的导航过程保驾护航。

AMCL

导航路线、带头司机、局部视野都有了，是否可以进行自动导航了？还不行，我们还缺定位器。没有定位器，就无法知道机器人在地图的哪个位置，还是无法按地图自动导航。

在 Navigation 中，作为定位器功能的是由一个叫 AMCL 的节点来完成的。AMCL 叫“蒙特卡罗粒子滤波定位算法”，为了便于理解，我们以“分身”来打比方。

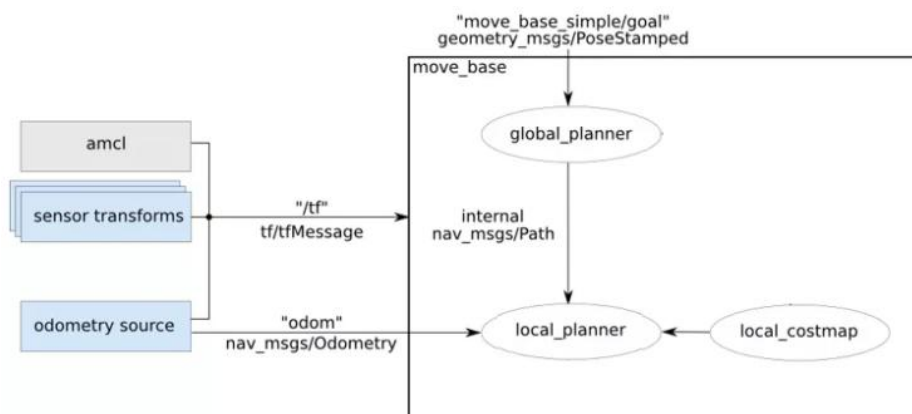
在我们的机器人开始移动之前，AMCL 会在机器人周围分裂出一堆的分身，这些分身随机的分布在地图里。在 Rviz 窗口里，我们会看到机器人周围有很多绿色的小箭头，这些小箭头就是 AMCL 变出来的“分身”。



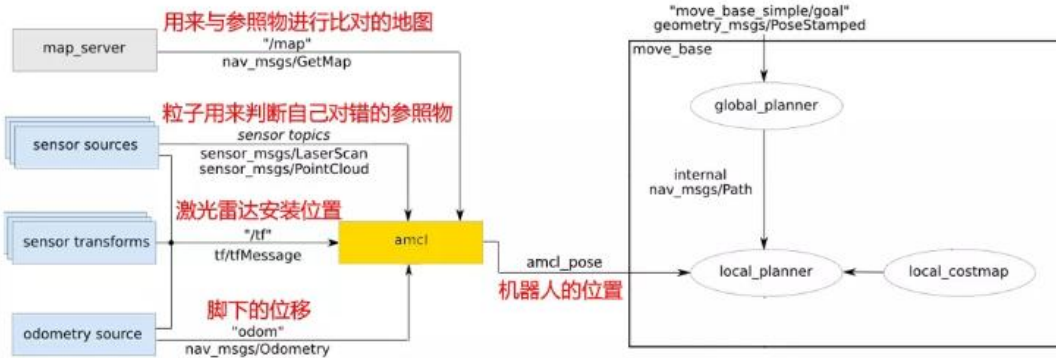
当机器人移动的时候，所有分身会用统一的步伐行进，这个统一步伐来自底盘电机码盘（odom 里程计），当电机码盘里程计提示机器人往前直行的时候，所有分身都会同时往前直行；当电机码盘里程计提示机器人往左转的时候，所有分身也同时往左转。

机器人的分身在移动的过程中，会不停的用激光雷达扫描到的身边障碍物和地图进行比对，以判断自己是不是那个正确的位置。如果不是，这个分身就会消失。随着机器人的不断移动，那些定位错误的分身一个接一个的消失，最后剩下的，就是最有可能是机器人位置的真身。对应的，在 Rviz 里，我们会看到导航中的机器人，身边的绿色箭头会逐渐收拢，最终聚合到机器人身上，和机器人真实的位置合为一体。

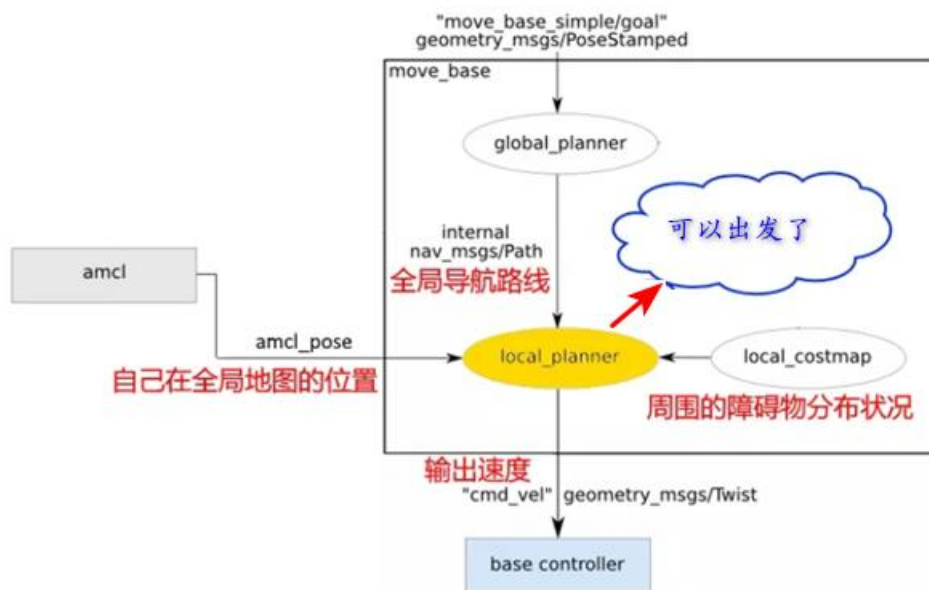
在 Navigation 架构图中，AMCL 的部分在最左侧：



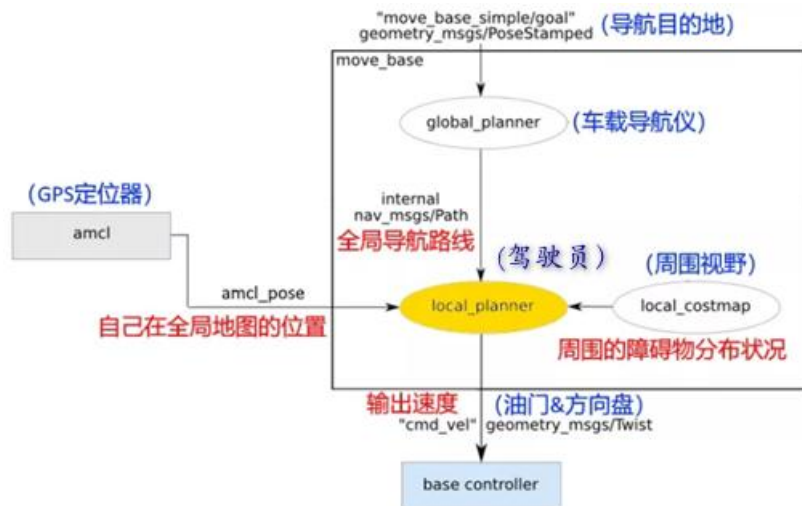
为方便理解，我们把原图调整一下：



加上 AMCL 定位器之后，我们的局部规划器（local_planner）终于可以自动导航了：



所以可以看到，Navigation 的结构虽然复杂，但是其目标还是很统一的。所有的一切，最终的目的就是让局部规划器（local_planner）能够正常自动导航。在整个 Navigation 系统里，局部规划器（local_planner）至关重要。如果把机器人比作一辆车，那么 Navigation 里的各部分模块所扮演的角色就像下图这样：



在实际的操作当中，对局部规划器（local_planner）的调教是最曲折最费时的部分。

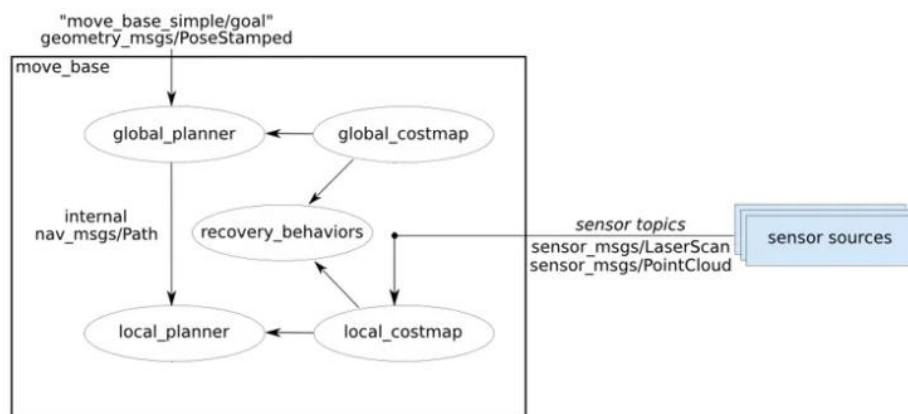
因为机器人的底盘类型千差万别，这导致很难用一个统一的局部规划器去适配所有的机器人。

这也使得在 ROS 中，局部规划器的类型是繁多，比如：base_local_planner、DWA、TEB、Eband 等。

异常恢复机制

在导航调试中，最常见的问题是机器人遇到新出现的障碍物时，会不停的原地打转，这

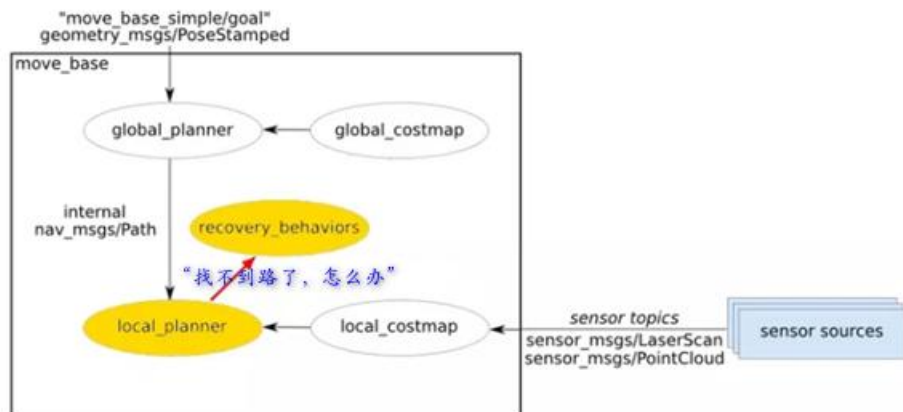
涉及到“recovery_behavior”的机制，在 Navigation 架构图中，它是这个样子的：



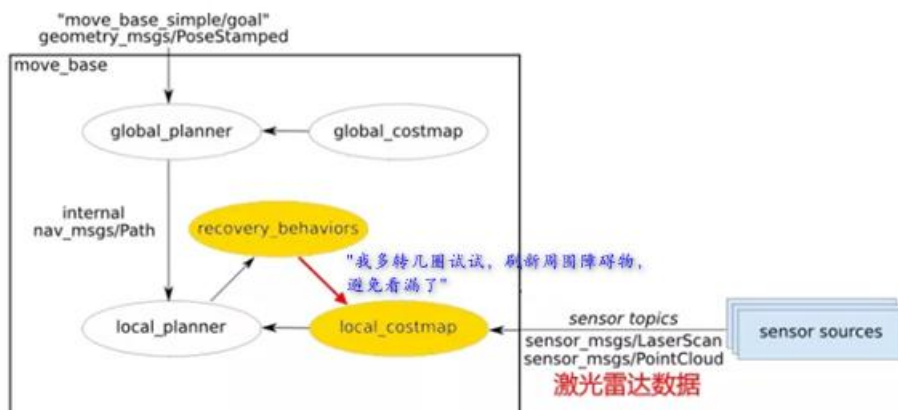
为方便理解，我们分成多个步骤来看：

（1）“recovery_behavior”被激活，通常是出现在机器人面前被大体积的障碍物挡住导航路线的时候。一些局部规划器（local_planner）如果参数设置不合理，就会导致规划不

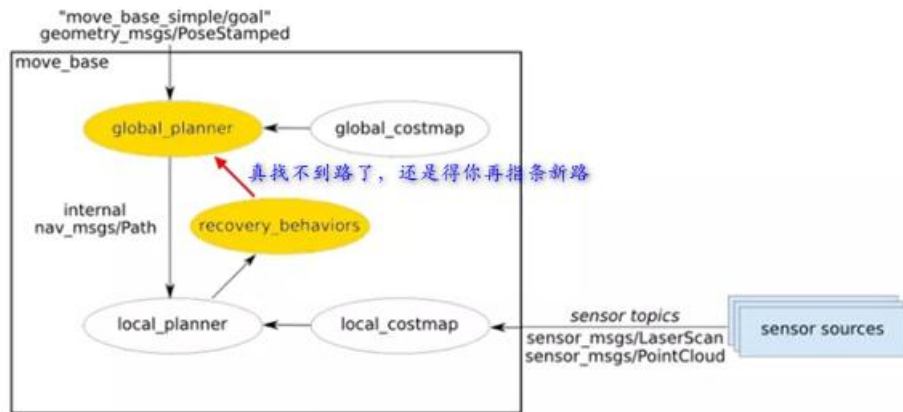
出能绕过障碍的局部路线，于是不得不向“recovery_behaviors”求助。



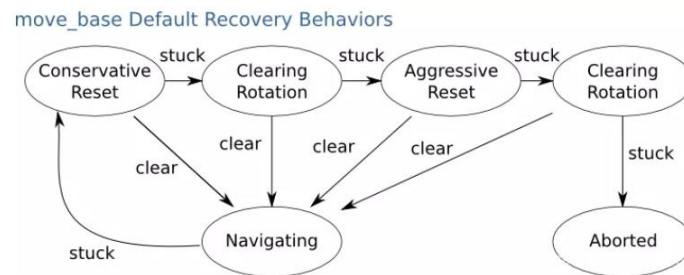
(2) 在很多版本的 ROS 中，“recovery_behaviors”里默认行为通常会设置为“rotate_recovery”，也就是让机器人原地旋转，用激光雷达去完全的扫描附近的障碍物。为何原地旋转就能完全扫描附近的障碍物？因为大部分的机器人，在激光雷达的周围，通常都会有一些支撑结构会挡住它的扫描视野，也就导致机器人的激光雷达在视野上有一些盲区。“recovery_behavior”觉得也许找不到路线只是因为局部规划器（local_planner）的视野被挡住了，能够绕过障碍物的路线搞不好就藏在激光雷达的视野盲区里，说不定转个一两圈就能解决问题了。毕竟让全局规划器（global_planner）重新规划全局路径太耗时了，如果可能，尽量不去做全局路径规划。



(3) 按照“recovery_behavior”设计的机制，当转圈也解决不了问题的时候，最终还是得全局规划器（global_planner）出马重新规划一条新的全局导航路线。



(4) 完整的“recovery_behaviors”状态链如下：



这么多状态，一不小心就在某个状态就“clear”到“Navigating”，然后又“stuck”卡回到“recovery_behaviors”状态链。于是，在这几种状态中不停的循环跳不出来，这个想让我们能够恢复导航状态的“recovery_behaviors”机制，可能让我们再也无法恢复到导航状态。

所以，对于不熟悉局部规划器（local_planner）参数的同学，建议在 launch 文件里直接把 move_base 的 recovery_behavior_enabled 参数设置为 false，直接禁掉好了。这样机器人遇到局部规划器解决不了的情况时会停下来，不会莫名其妙的转圈，干扰用户对机器人运行状态的判断。

2

实验原理

1. 使用 ROS 中的 Navigation 导航包，基于 2D 栅格地图进行路径规划与导航。

2. 导航过程中使用 AMCL “蒙特卡洛粒子滤波定位算法” 进行机器人定位。
3. 导航过程中，局部规划器使用局部代价地图避免发生碰撞。
4. 导航路线上遇到新障碍物，局部规划器尝试动态调整与纠偏，寻找新路线绕开障碍物。
5. 导航过程中遇到障碍物时的异常恢复机制，尝试让机器人继续完成导航达到终点。
6. 当有新障碍物时，如果局部规划器找不到路径，会从全局规划器中重新规划路径，以达到终点。

3

实验目的

1. 了解 Navigation 功能包的架构及构成。
2. 了解全局规划器、局部规划器、代价地图、AMCL 等概念。
3. 了解自动导航的异常恢复机制
4. 掌握基于 2D 栅格地图的路径规划与导航原理及过程。
5. 掌握在仿真环境中的进行自动导航仿真。

4

实验环境

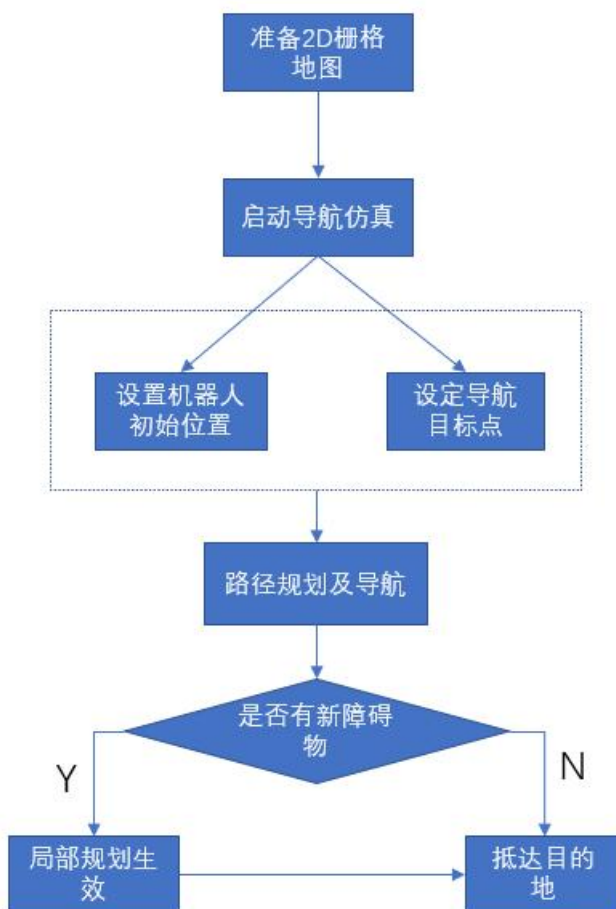
1. 代码开发工具版本：Visual Studio Code 1.89.0 及以上

2. 测试工具: rviz、Gazebo
3. 操作系统版本: Ubuntu20.04
4. 机器人操作系统 ROS 版本: Noetic
5. Git 版本: 24.0.0 及以上
6. 开发语言: Python 3.8 及以上 , C++ 11 及以上

5

实验流程

实验整体流程图如下所示:



- (1) 通过上一节的激光 SLAM 建图，获得 2D 栅格地图，放入自动导航功能包的指定目录作为地图。
- (2) ROS 中启动导航仿真 Launch 文件。
- (3) 在 RViz 中设置机器人初始位置，使得地图上的位置、朝向和仿真环境中的基本一致。
- (4) 在 RViz 中设定导航目标点。
- (5) 在 RViz 和 Gazebo 中观察自动导航的过程及结果。
- (6) 进行多次测试，在 Gazebo 中在导航路线上放置障碍物，观察局部规划避障的行为。

6

实验任务

1.1.1 任务一 Navigation 导航仿真

1. 将上一节激光 SLAM 建图结果文件导入

上一节的 Gmapping 建图结果默认保存在 Home 目录：

```
ros@ubuntu:~$ cd $HOME  
ros@ubuntu:~$ ls map*  
map.pgm  map.yaml
```

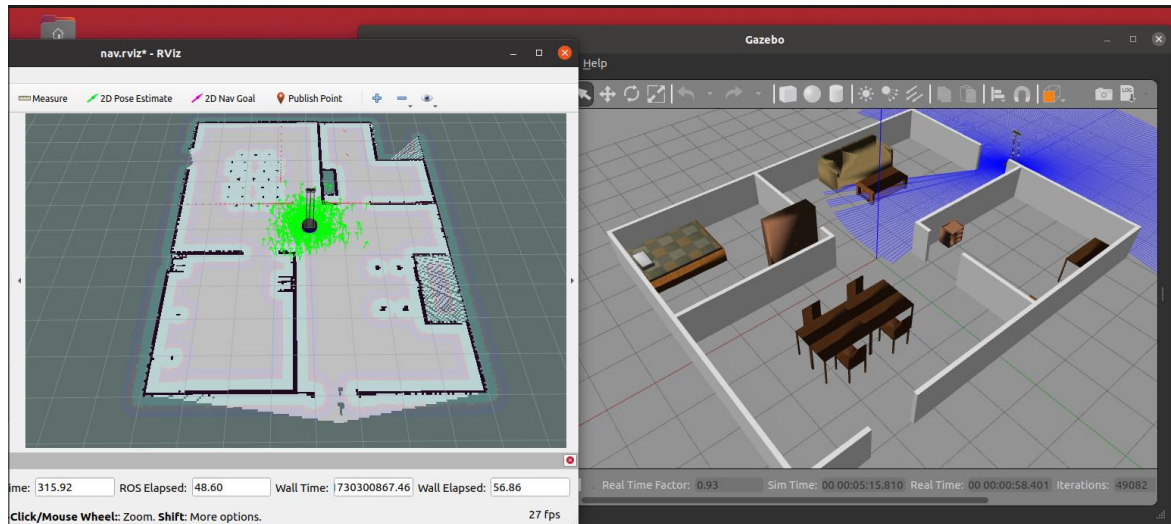
将上述两个文件复制到导航功能包的 maps 目录中：

```
ros@ubuntu:~$ cp map.* ~/catkin_ws/src/wpr_simulation/maps/
```

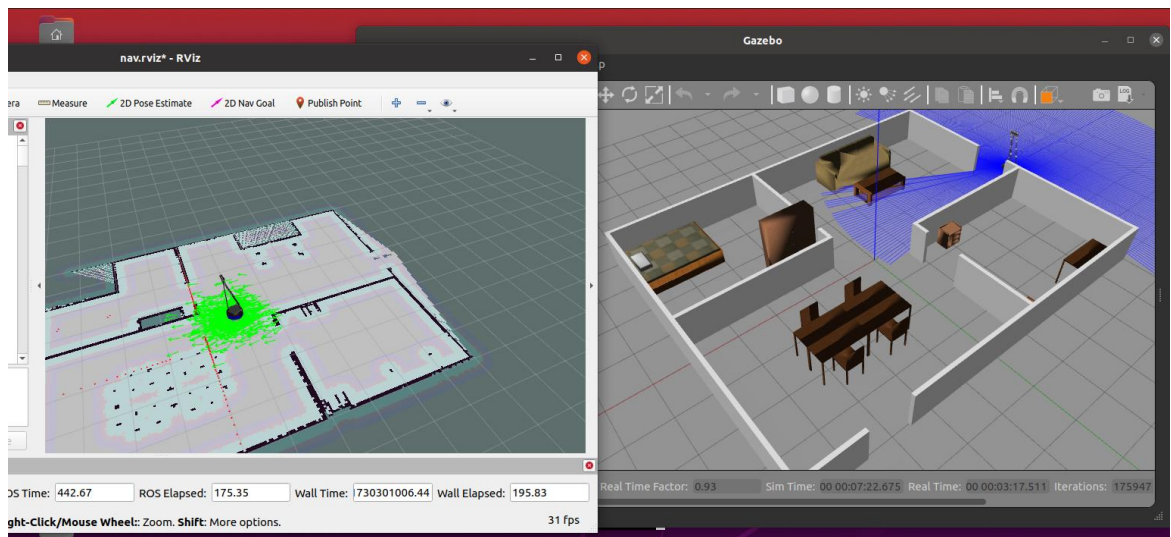
2. 输入指令，启动导航节点

```
roslaunch wpr_simulation navigation.launch
```

系统会启动 Gazebo 窗口和 Rviz 窗口：



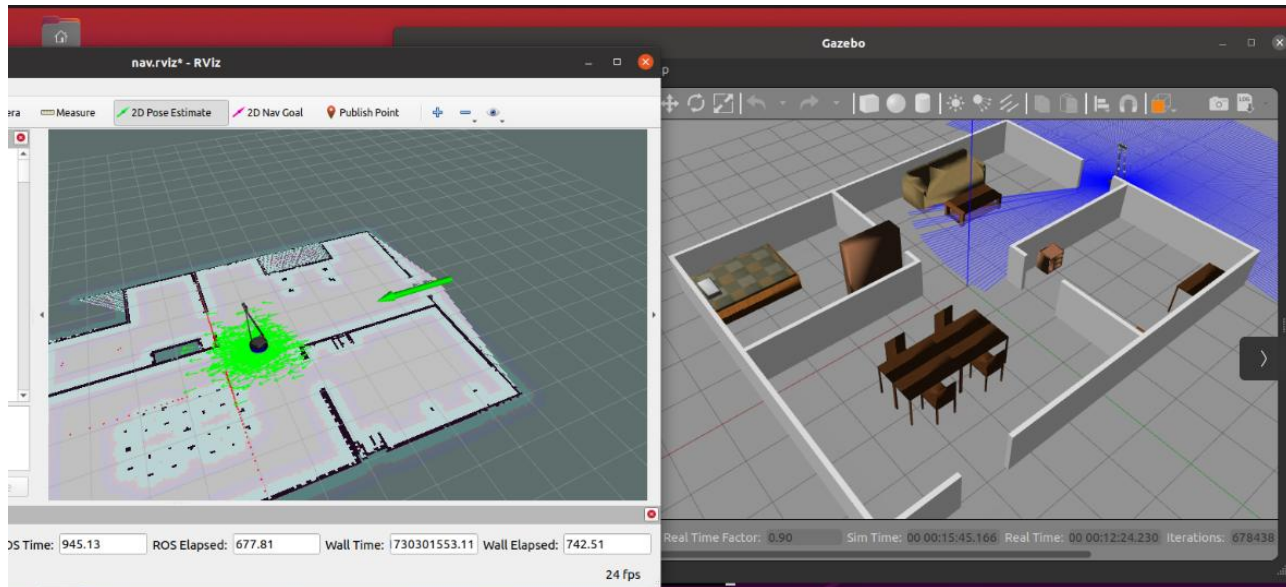
将两个窗口调整到合适的位置和大小，并调整地图方向，确保 rviz 中地图和 Gazebo 仿真环境的方向一致，方便观察：



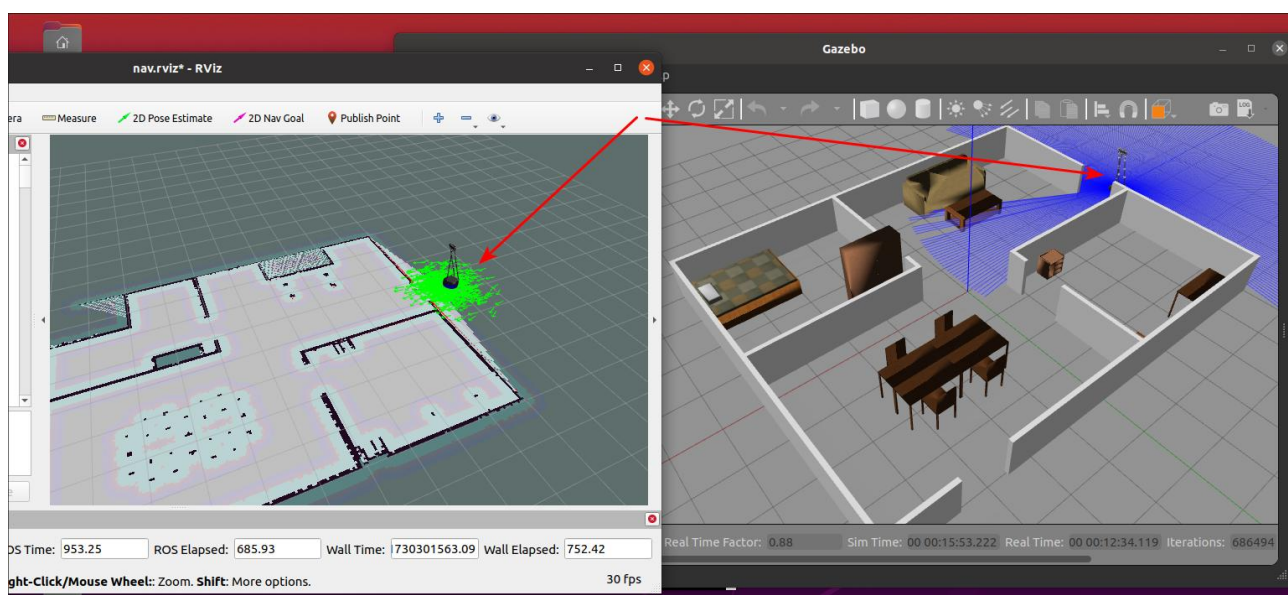
可以看到 Rviz 中的地图，障碍物周围出现浅蓝色的安全边界，这是代价地图在起作用，避免机器人在导航运动过程中和障碍物发生碰撞。机器人周围有很多绿色箭头，这是 AMCL 蒙特卡洛粒子滤波定位算法在起作用。

3. Rviz 中设置机器人初始位置

当前 Rviz 中机器人位于地图中央，而 Gazebo 环境中，机器人位于门口朝向门内。在导航前需要将 Rviz 中机器人设置到正确的位置，朝向也尽量保持一致。在 Rviz 窗口上方工具栏里有“2D Pose Estimate”按钮，点击该按钮，然后鼠标在 Rviz 地图中点击机器人应该处于的位置。

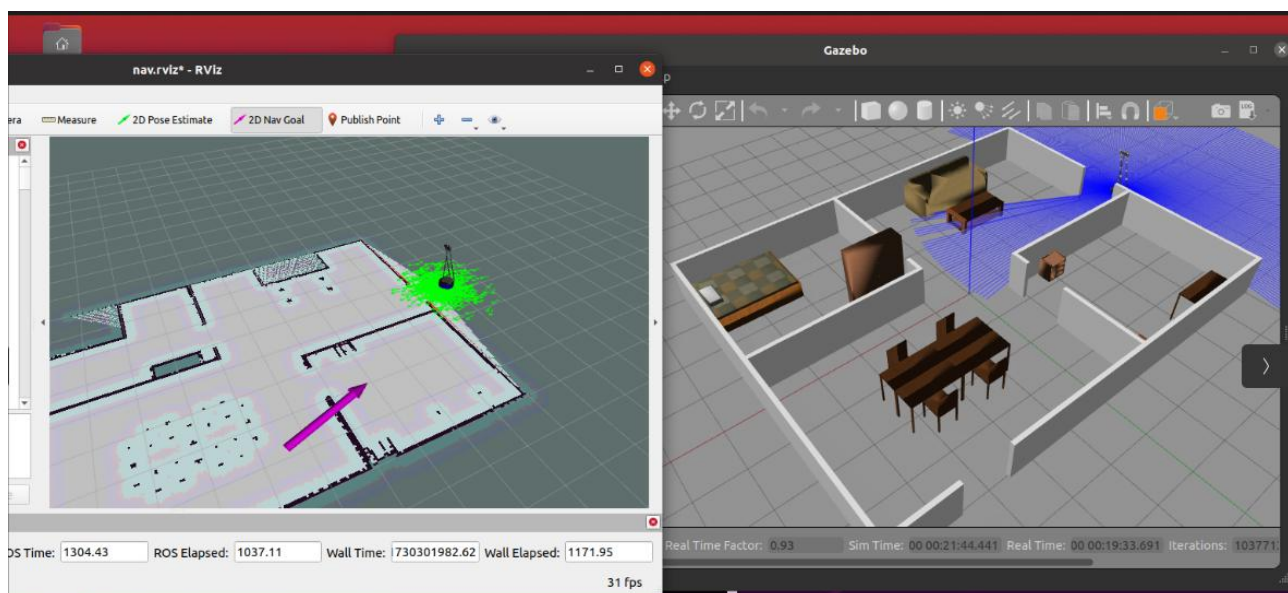


这时会出现一个绿色大箭头，代表的时机器人的初始位置朝向，按住鼠标左键不放，控制绿色箭头的朝向，直到和 Gazebo 中机器人大体一致的朝向时再松开鼠标左键。机器人模型就会定位到选择的位置和朝向。此时可以看到机器人朝向探测到的墙体的红色雷达数据点和墙基本吻合，说明位置设置合适，如果有明显偏差或者偏离，则需要重新定位机器人的位置和朝向。

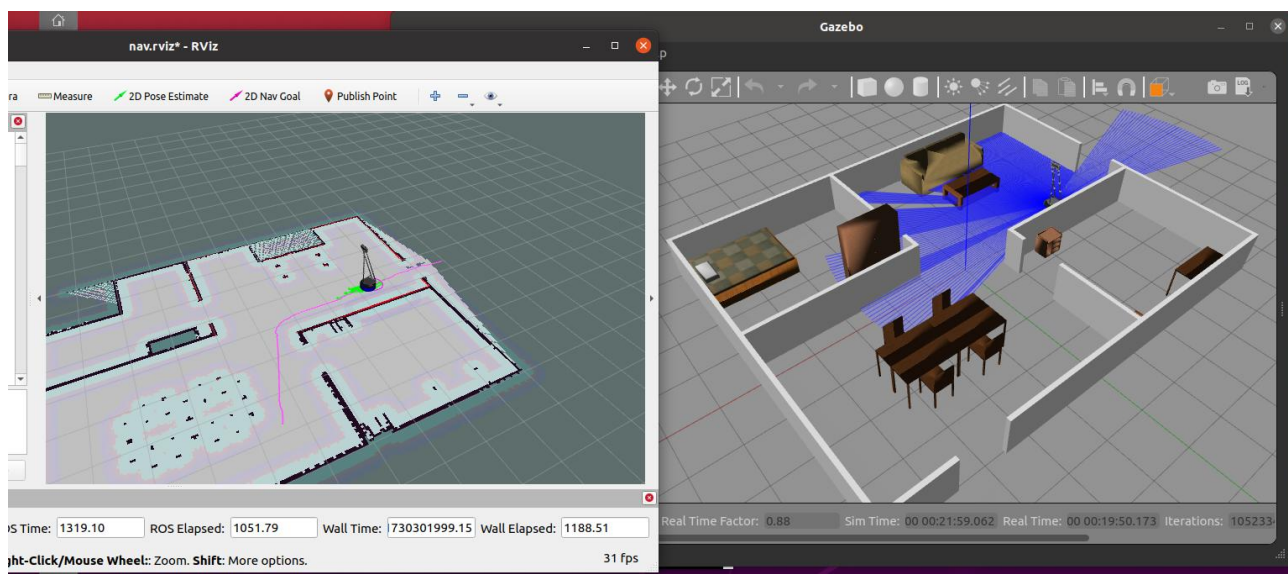


4. Rviz 中设置导航目标点

单击 Rviz 窗口上方工具栏里的“2D Nav Goal”按钮，然后单击 Rviz 地图上的导航目标点。此时会出现紫色箭头，按住鼠标左键在屏幕上调整箭头朝向来决定机器人到达目标点时的朝向。

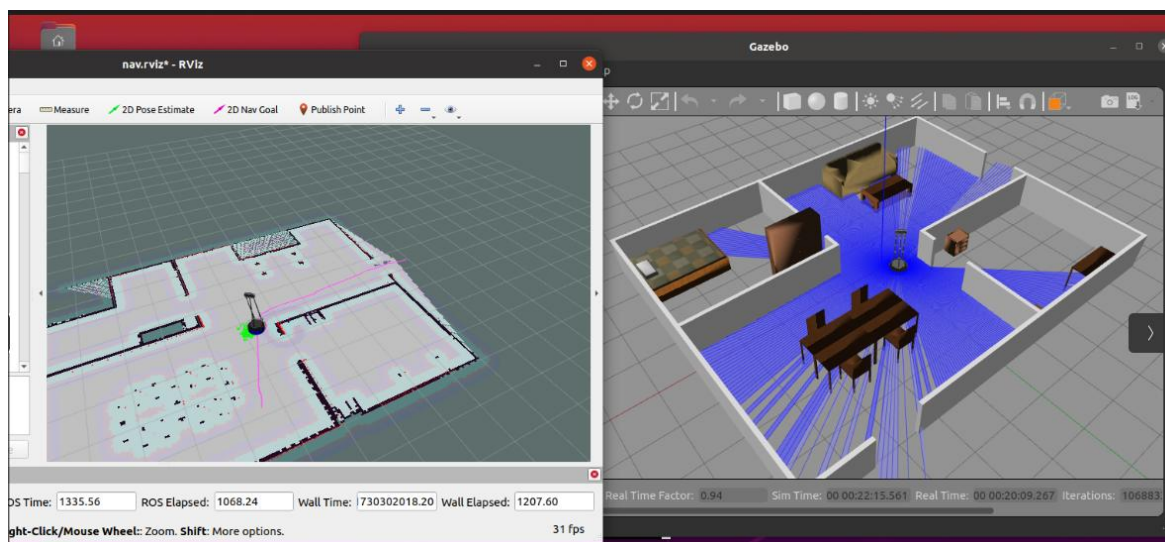


松开鼠标左键，ROS 的导航系统就会规划一条紫色的路径。这条路径会绕开代价地图的蓝色安全边界，一直到移动目标点结束。



5. 机器人按导航路径移动:

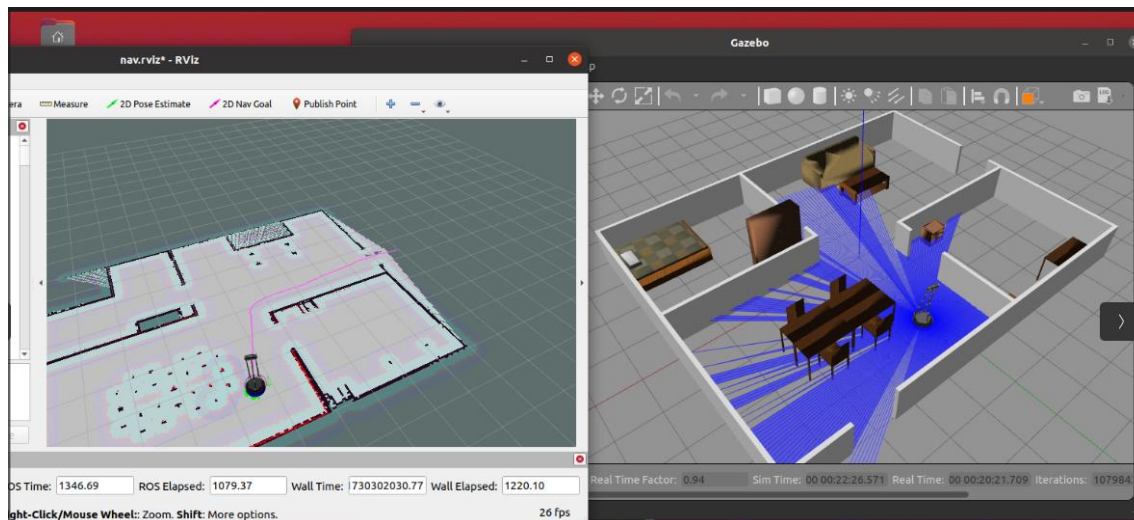
Gazebo 中，机器人移动的路径和 Rviz 中保持一致。在 Rviz 中可以看到机器人靠近障碍物时，会在浅蓝色代价地图周围又出现一层浅紫色的代价地图，这是本地规划器在根据实时扫描周围的最新情况进行观察应对，来保障导航能应对突然出现的障碍物等突发情况，确保导航成功。



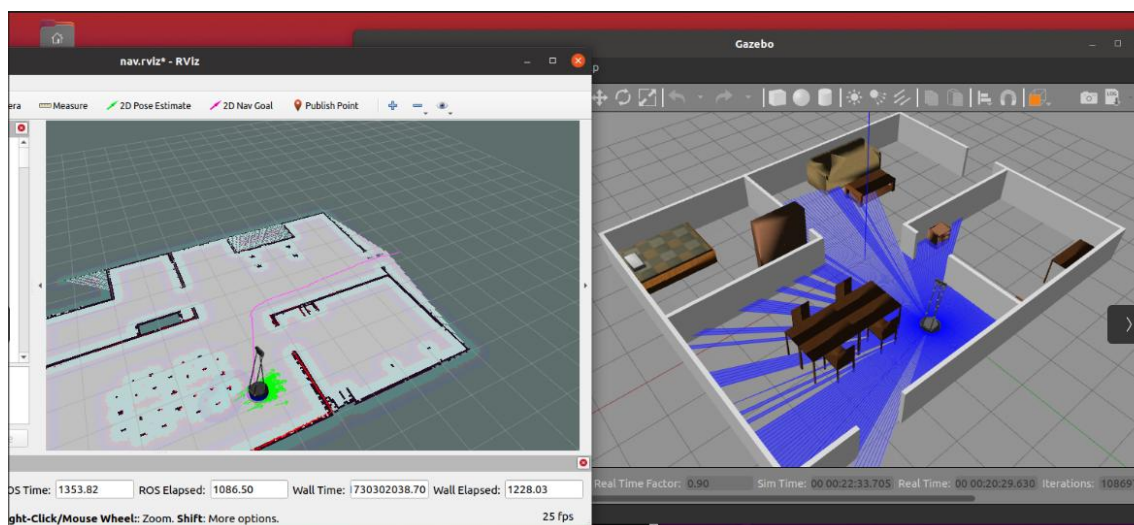
观测可以发现，机器人周围的 AMCL 粒子分身，随着机器人的移动会不断的消失和重新生成，但总体上是沿着规划路径的，从前面对 AMCL 粒子分身的介绍可以知道，每个分身都在不断扫描周围障碍物来判断自身是否在正确的位置，不在的话会自行消失，这样在正

确位置的分身会留存下来，这样分身不断生成、不断消亡，支持了对机器人的位置定位。

6. 机器人最终成功达到导航目标点



到达目标点后，机器人会原地旋转，调整航向角，朝向最初设定的朝向：

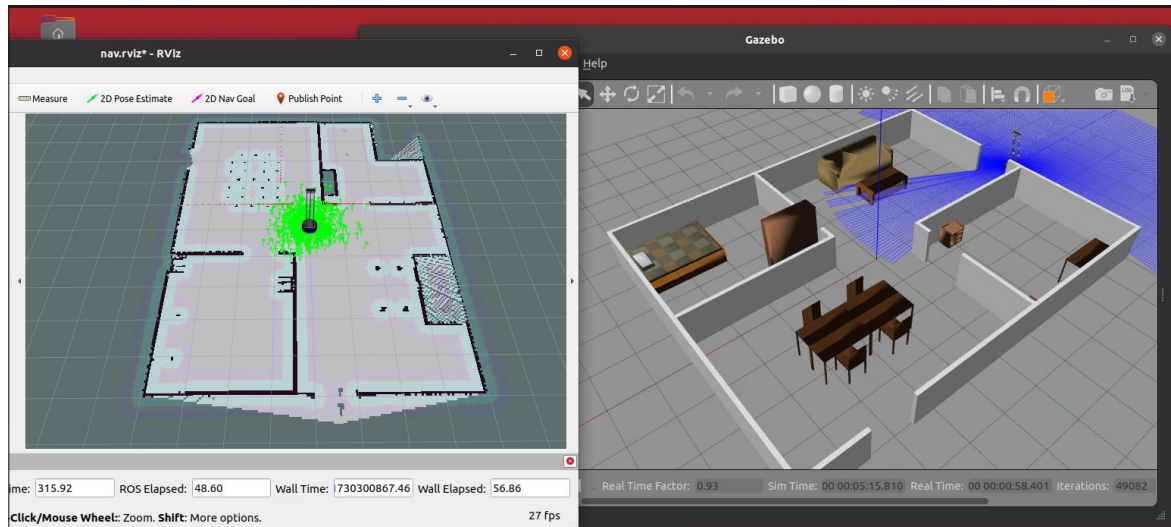


1.1.2 任务二 Navigation 导航仿真遇到挡路障碍物

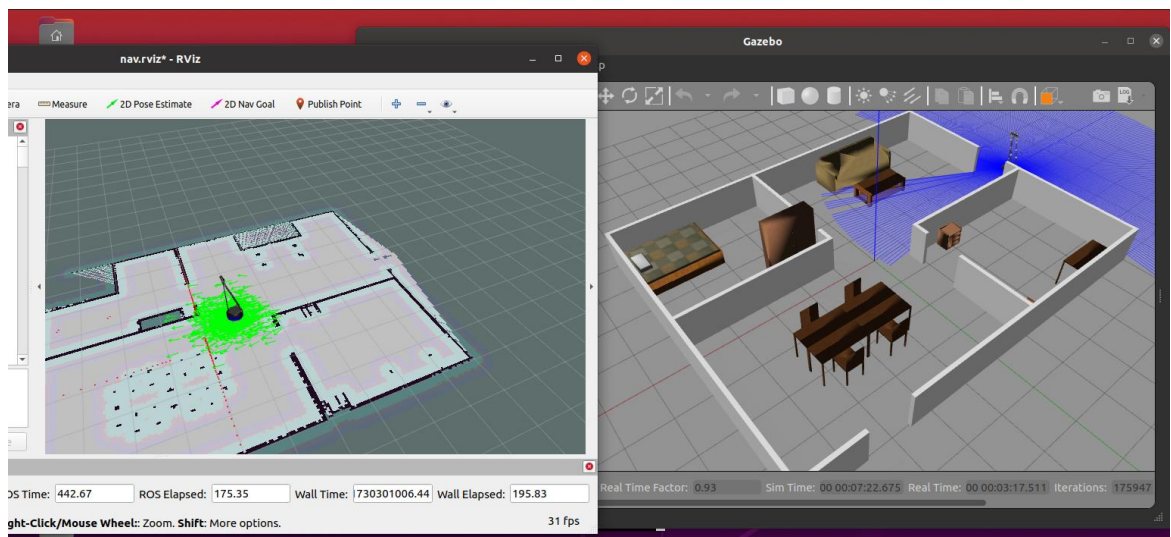
1. 输入指令，启动导航节点

```
roslaunch wpr_simulation navigation.launch
```

系统会启动 Gazebo 窗口和 Rviz 窗口：



将两个窗口调整到合适的位置和大小，并调整地图方向，确保 rviz 中地图和 Gazebo 仿真环境的方向一致，方便观察：

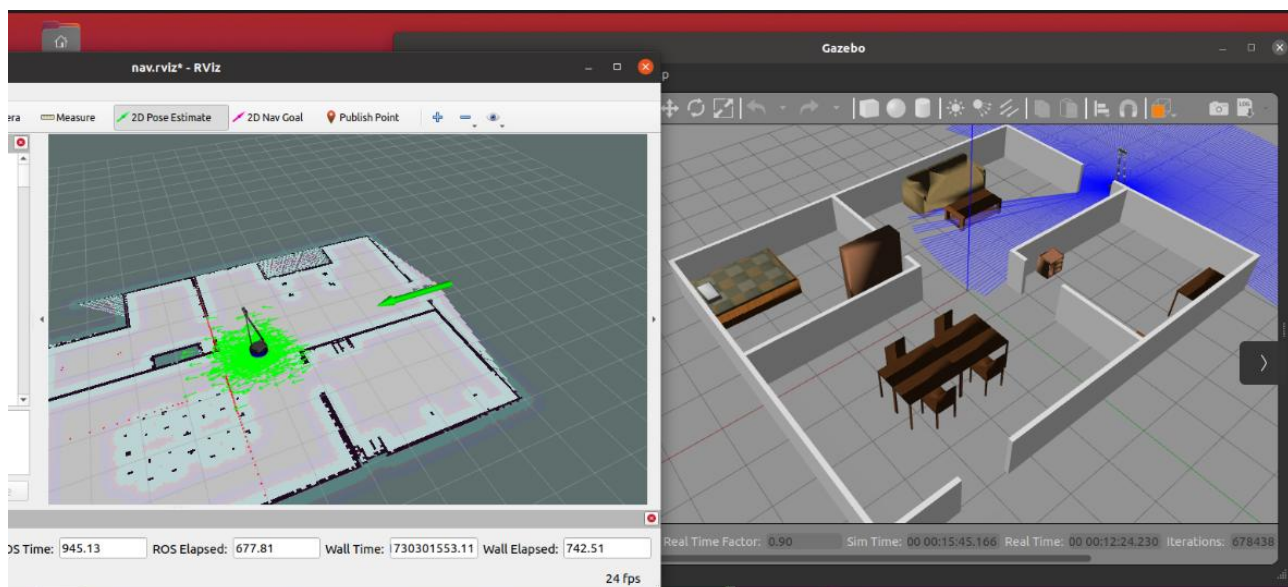


可以看到 Rviz 中的地图，障碍物周围出现浅蓝色的安全边界，这是代价地图在起作用，避免机器人在导航运动过程中和障碍物发生碰撞。机器人周围有很多绿色箭头，这时 AMCL 蒙特卡洛粒子滤波定位算法在起作用。

2. Rviz 中设置机器人初始位置

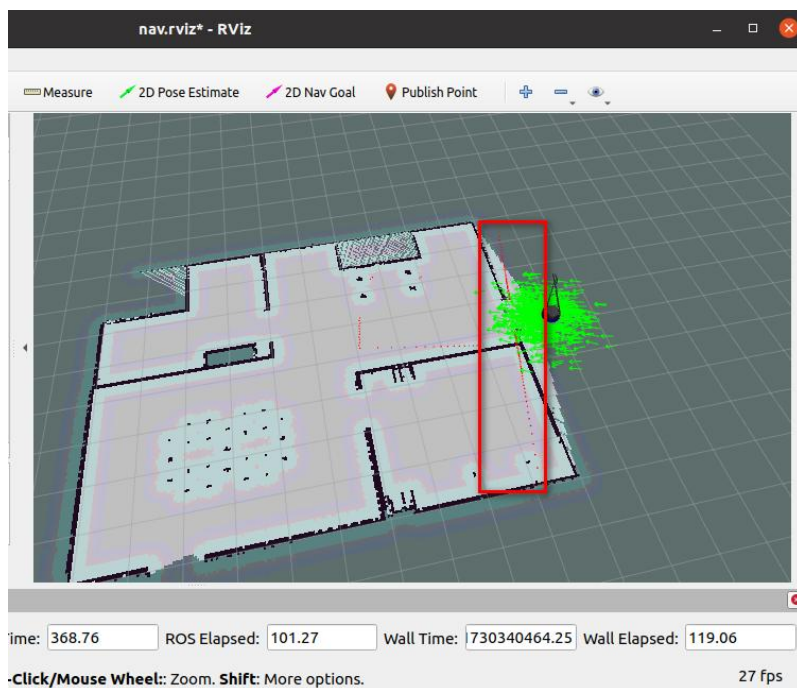
当前 Rviz 中机器人位于地图中央，而 Gazebo 环境中，机器人位于门口朝向门内。在导航前需要将 Rviz 中机器人设置到正确的位置，朝向也尽量保持一致。在 Rviz 窗口上方工

具栏里有“2D Pose Estimate”按钮，点击该按钮，然后鼠标在 Rviz 地图中点击机器人应该处于的位置。

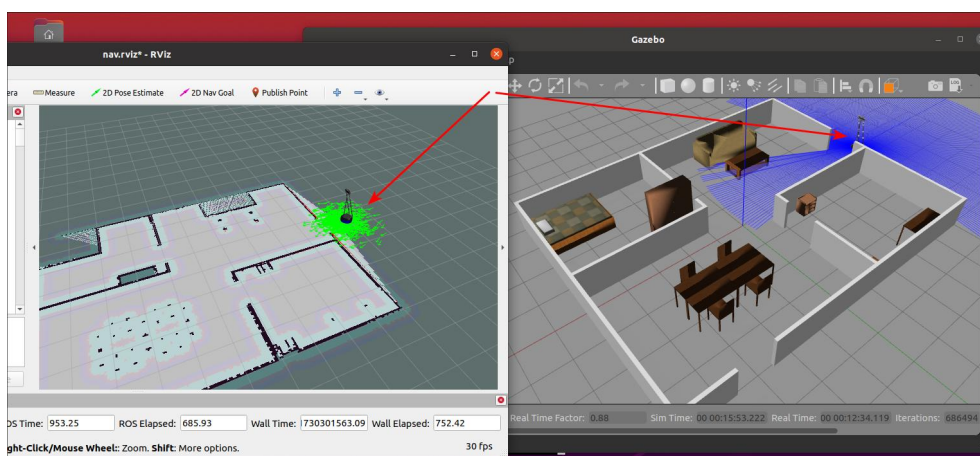


这时会出现一个绿色大箭头，代表的是机器人的初始位置朝向，按住鼠标左键不放，控制绿色箭头的朝向，直到和 Gazebo 中机器人大体一致的朝向时再松开鼠标左键。机器人模型就会定位到选择的位置和朝向。

如果有明显偏差或者偏离，则需要重新定位机器人的位置和朝向。比如下图这种情况，朝向设置明显不对，需要继续调整知道选中红线和墙体基本平行、重合。

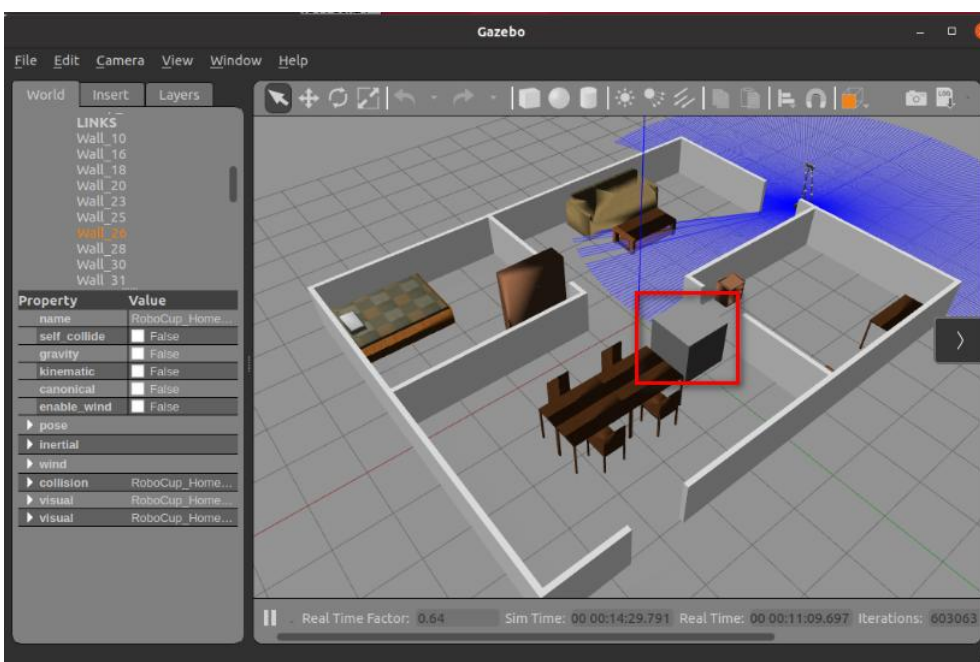
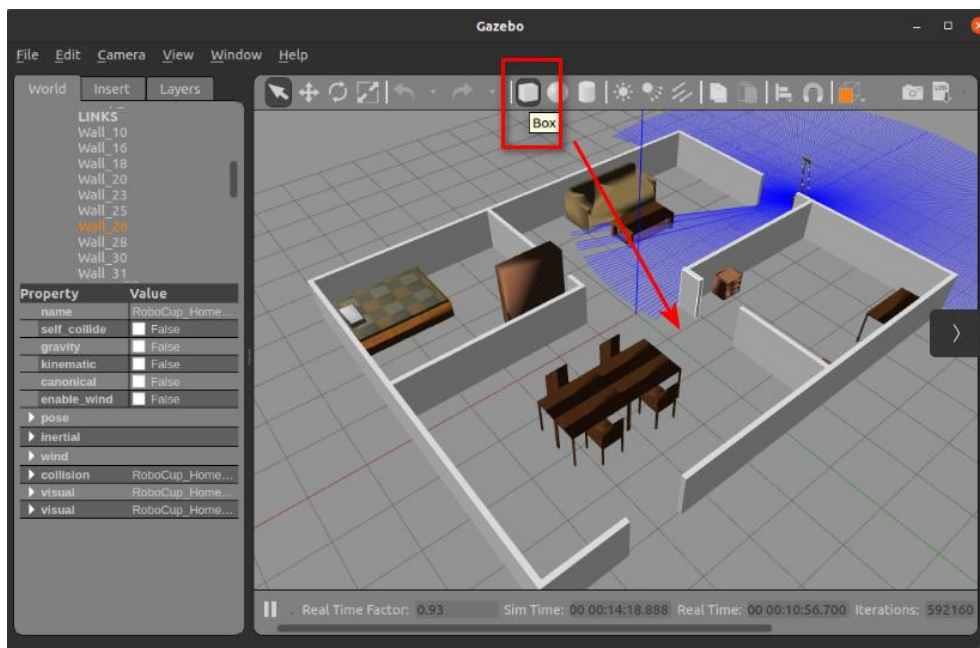


此时可以看到机器人朝向探测到的墙体的红色雷达数据点和墙基本吻合,说明位置设置合适。



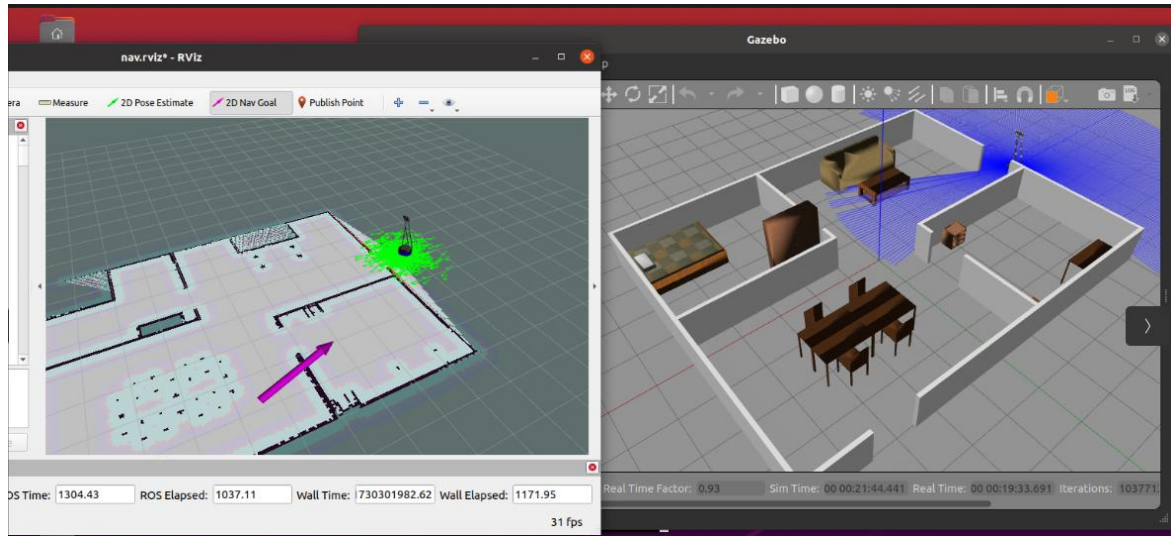
3. 在自动导航的线路上放置障碍物

在 Gazebo 窗口中, 选择上面工具栏的 Box 图标, 放置一个立方体到准备选定的自动导航路线上, 这样可以使得机器人在经过障碍物时被迫进行局部规划, 进行绕行或者改道。

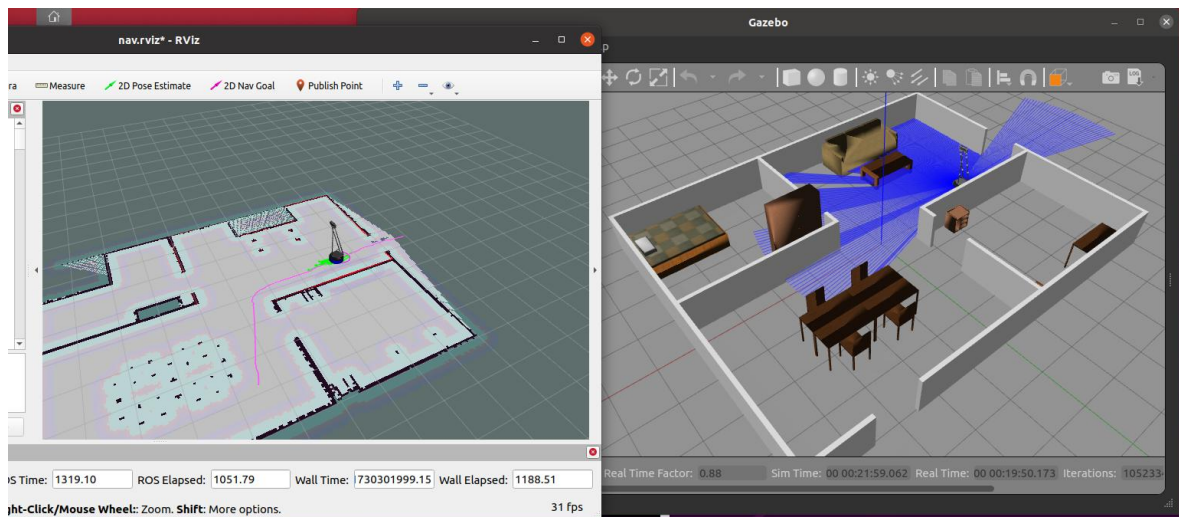


4. Rviz 中设置导航目标点

单击 Rviz 窗口上方工具栏里的“2D Nav Goal”按钮，然后单击 Rviz 地图上的导航目标点。此时会出现紫色箭头，按住鼠标左键在屏幕上调整箭头朝向来决定机器人到达目标点时的朝向。

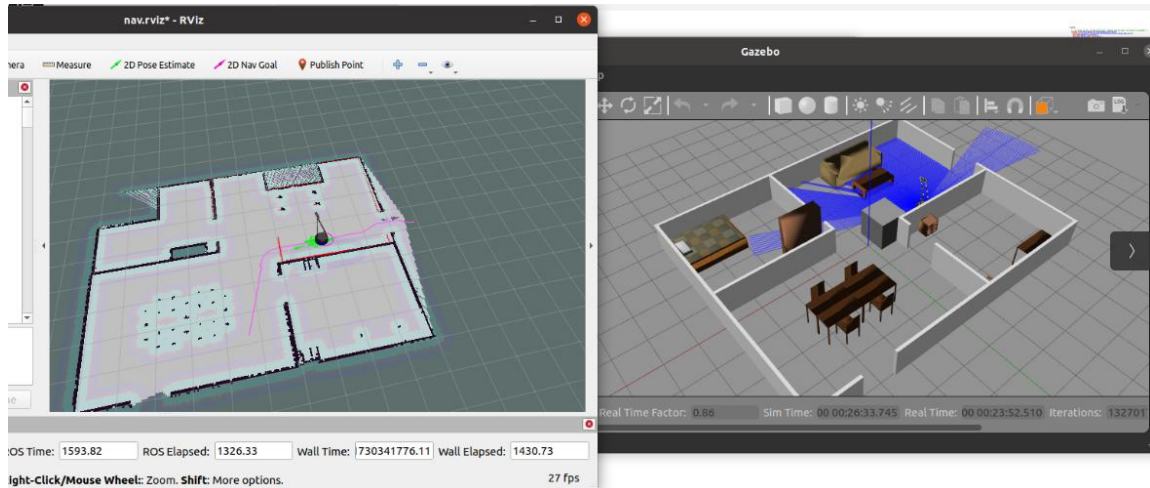


松开鼠标左键，ROS 的导航系统就会规划一条紫色的路径。这条路径会绕开代价地图的蓝色安全边界，一直到移动目标点结束。



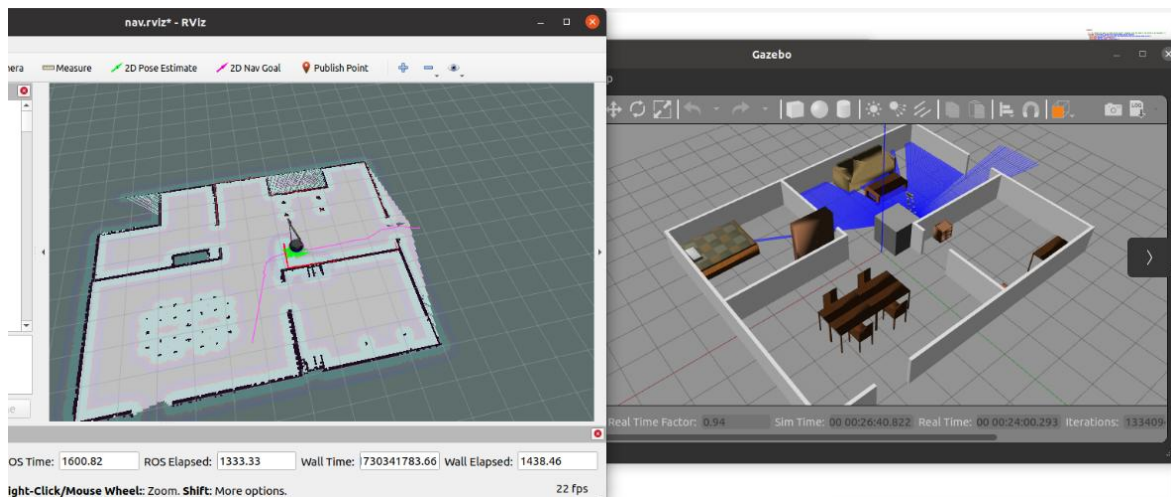
5. 机器人按导航路径移动：

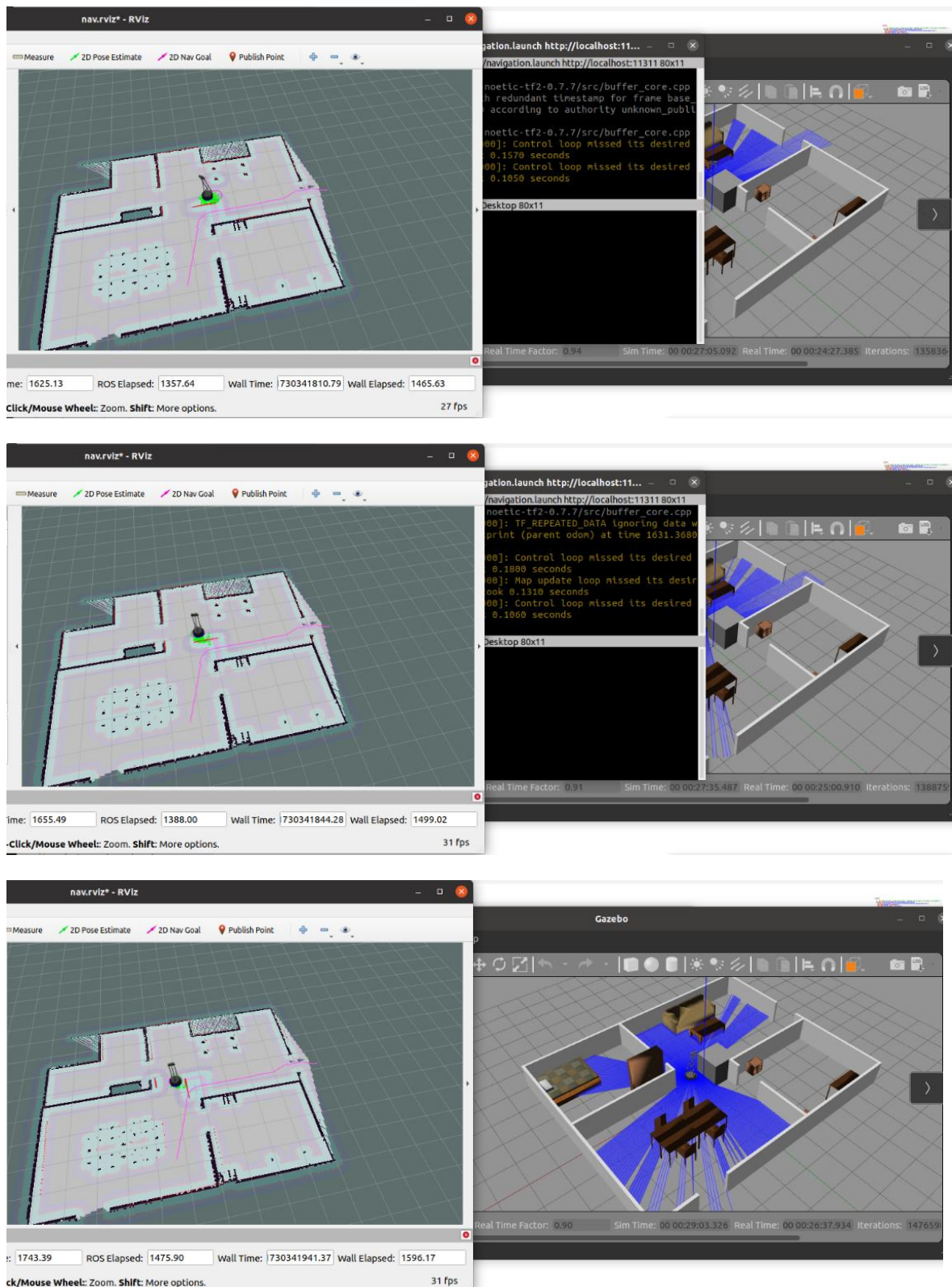
Gazebo 中，机器人移动的路径和 Rviz 中保持一致。在 Rviz 中可以看到机器人靠近障碍物时，会在浅蓝色代价地图周围又出现一层浅紫色的代价地图，这是本地规划器在根据实时扫描周围的最新情况进行观察应对，来保障导航能应对突然出现的障碍物等突发情况，确保导航成功。

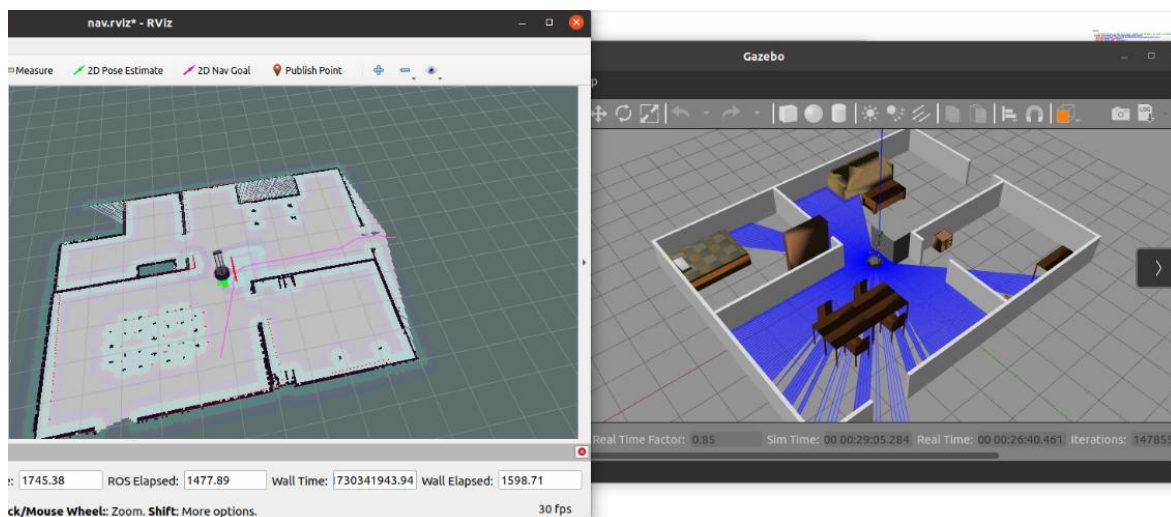


观测可以发现，机器人周围的 AMCL 粒子分身，随着机器人的移动会不断的消失和重新生成，但总体上是沿着规划路径的，从前面对 AMCL 粒子分身的介绍可以知道，每个分身都在不断扫描周围障碍物来判断自身是否在正确的位置，不在的话会自行消失，这样在正确位置的分身会留存下来，这样分身不断生成、位置不对的分身不断消亡，支持了对机器人的位置定位。

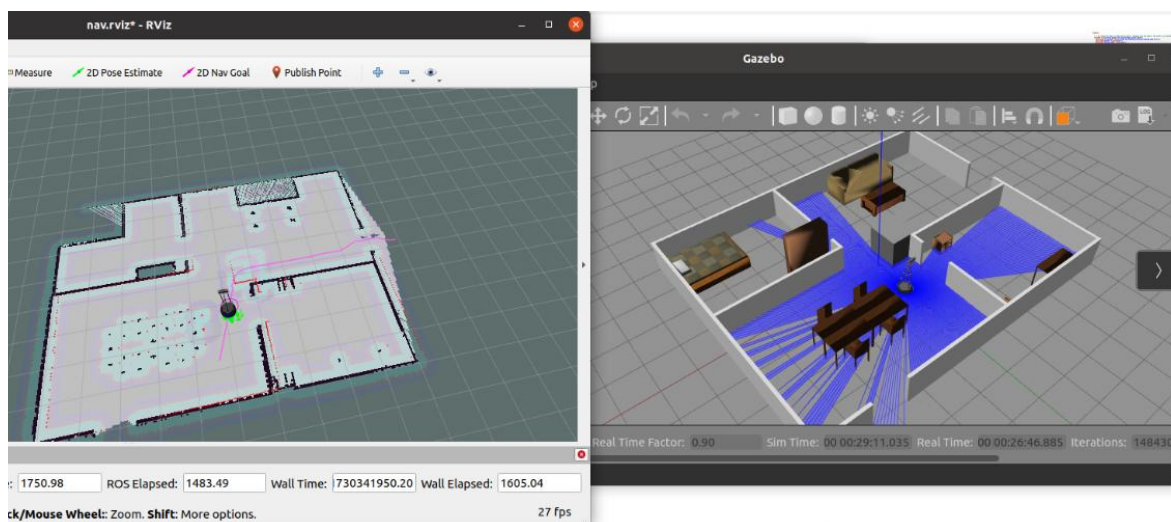
当机器人运行到放置的立方体障碍物跟前时，局部规划器会发现障碍物，并尝试绕开：



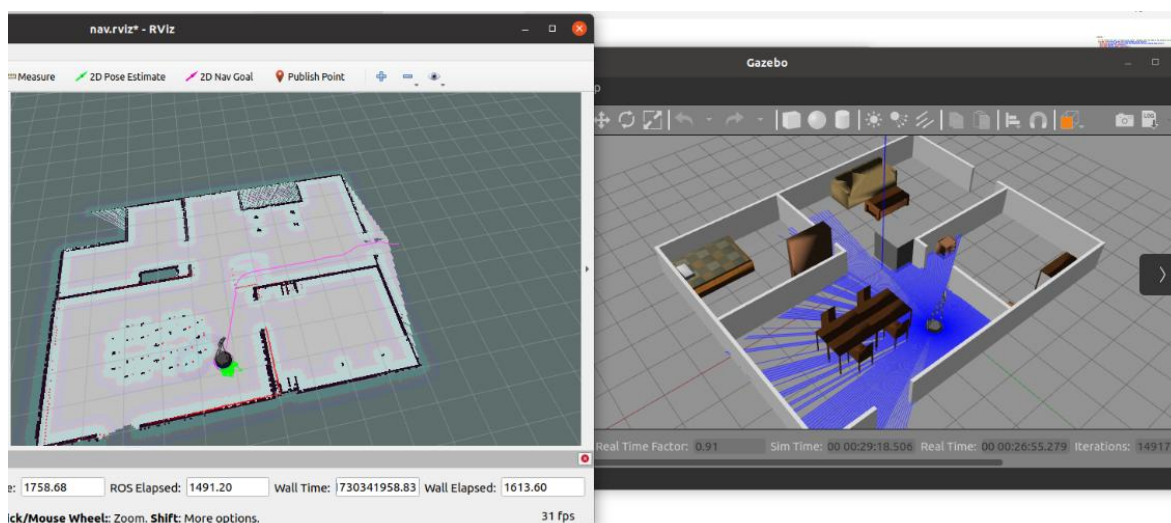




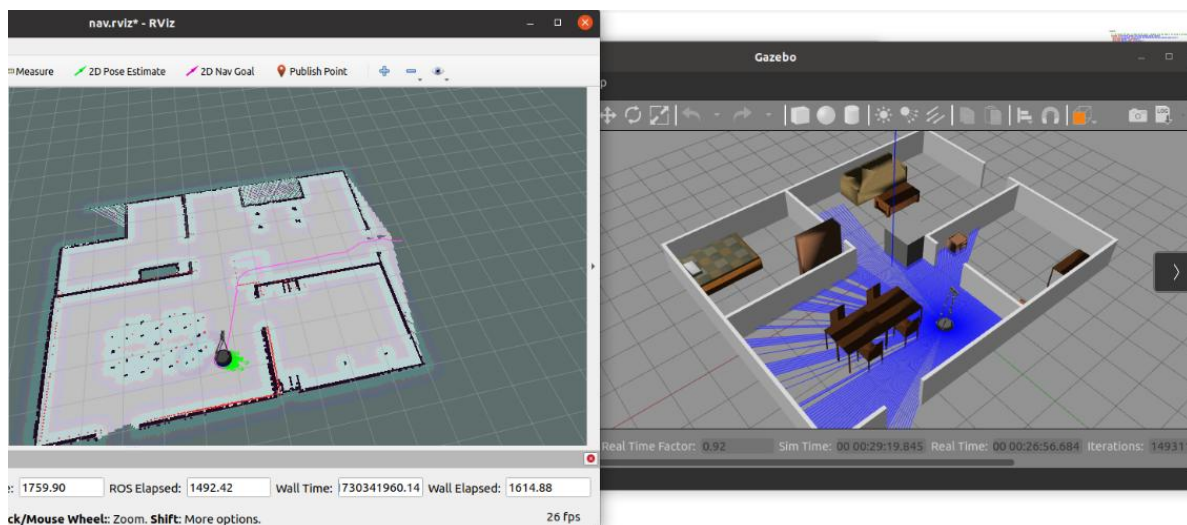
绕开障碍物后，机器人又回到原来规划的全局路线上来。



6. 机器人最终成功达到导航目标点



到达目标点后，机器人会原地旋转，调整航向角，朝向最初设定的朝向：



7

实验结果

1. 学会了如何启动 iCar 智能小车及通过 APP 操控 iCar 智能小车的底盘运动。
2. 学会了如何进入 ROS2 的 docker 开发环境。
3. 学会了在 ROS2 的 docker 环境中启动运行雷达避障应用节点。
4. 学会了代码上传到 ROS2 的 docker 环境中并重新编译运行。
5. 学会了如何用代码控制 iCar 智能小车的运动。
6. 学会了如何开发部署雷达 ROS2 应用，获得并使用雷达测距扫描数据。
7. 学会了如何编写、修改雷达避障应用代码。

8

实验资源释放

1. 关闭虚拟机上的 ROS 程序和终端窗口。
2. 关闭虚拟机。
3. 关闭宿主机。

9

实验小结

通过该实验,我们了解了 ROS 中的 Navigation 功能包的架构构成及各功能组件的概念、原理、作用,学习了自动导航的异常恢复机制,并进行了基于 2D 栅格地图的路径规划与导航的仿真实验,观察了整个导航过程,及过程中局部规划器如何根据实际路况进行动态规划、调整、纠偏、避障的。

该实验使我们对自动驾驶导航的工作原理及过程有了深入而具体的掌握和感知。